



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS organizes the frontier based on only $g(n)$, the total cost so far, and so proceeds outward from the origin without knowing anything about where the goal state is, in concentric rings. A*, on the other hand, organizes the frontier based on $f(n) = g(n) + h(n)$, where $h(n)$ is a cost-to-go heuristic estimate that introduces bias towards the goal state. Because of this, A* searches through fewer states than UCS to find the optimal path.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

The property of being admissible indicates that the estimated remaining cost should not exceed the real cost at any point in time. The minimal number of moves that are needed to travel from cell to cell without considering any walls in a 4-neighbor unit-cost grid is simply $|\Delta x| + |\Delta y|$. The walls can never shorten the path, hence the estimation is always less than or equal to the real cost.

Should the estimation exceed the real cost instead, A* would be able to evaluate a point that is situated on the optimal path as less valuable than some other point that is on a different, longer path. The algorithm would terminate and choose a longer path to reach the goal without even trying the optimal one. Thus, it would start acting in a greedy manner similar to best-first search.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No. Since both diagonal and orthogonal movements have equal costs, a single diagonal move corresponds to one unit of movement along both x and y axes. The actual shortest number of movements from one point to another is hence $\max(|\Delta x|, |\Delta y|)$. The Manhattan Distance ($|\Delta x| + |\Delta y|$) is now always an overestimate except for cases where both points are on the same axis, say (0,0) to (3,3). The actual cost will be 3 diagonal moves while Manhattan distance will give 6 moves. Manhattan Distance will thus not satisfy the lower bound criterion and will not be admissible anymore. Use `manhattan_distance = max(abs(x1-x2), abs(y1-y2))` instead.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

The "distance to nearest food" heuristic is bad because it is myopic – it does not take into account the total travel remaining once that pellet has been consumed. This allows the agent to simply hop around from pellet to pellet in a way that may look good locally but will be highly inefficient in aggregate (basically a travelling salesman problem without any optimization).

Instead, we could use a "minimum spanning tree" heuristic. We would calculate the MST of the set {current agent location} $\cup$ {remaining pellets}, taking into account the Manhattan distance as edge weights, and the total edge weight of this tree as $h(n)$. This heuristic is admissible, because any path that touches all the remaining pellets is necessarily greater in cost than the minimum cost structure that spans all the remaining points (the MST), which therefore provides a lower-bound estimate on the true remaining cost, unlike the greedy approach mentioned above.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

Incomplete: {filledCount}/{fields.length} questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING